

6. Ejercitación Clase 8

Clases y métodos abstractos

Sobre esta guía:

Lea atentamente cada uno de los enunciados y antes de llevar a código cada ejercicio realice en papel o con la ayuda de algún software, el correspondiente diagrama de UML, detallando los atributos, métodos y constructores que considere necesarios. Una vez terminado escriba el código y ejecute las pruebas que se detallan.

Enunciados:

1. Modela un sistema que gestione los datos de los empleados de una empresa.
 - a. Se contará con una **clase padre Empleado de la cual no se podrán crear objetos**. Cada empleado tiene un nombre y un método calcularPago() que debe ser implementado de manera específica para cada tipo de empleado (utilizando polimorfismo)
 - b. Implementa las siguientes clases hijas de la clase Empleado: **EmpleadoTiempoCompleto**, **EmpleadoPorHoras**, **EmpleadoAcomision** y **EmpleadoContratista**
 - c. Los empleados a tiempo completo tienen años de antigüedad y un sueldo básico. En ellos el método calcularPago() debe retornar ese salario mensual más el monto extra que corresponda por antigüedad, el cual será:
 - menos de dos años salario base;
 - de 2 a 5 años: 5% más;
 - de 6 a 10 años: 10% más;
 - de 11 a 15 años: 15% más;
 - y más de 15 años: 20% más.
 - d. Para los empleados por horas, el método calcularPago() debe retornar un salario calculado según las horas trabajadas y el precio que se le paga la hora a dicho empleado.
 - e. Los empleados contratistas son empleados contratados para proyectos específicos (su trabajo comienza y termina con dicho proyecto). el método calcularPago() debe retornar un salario calculado según el número de horas trabajadas y la tarifa por hora asignada a ese proyecto.
 - f. El pago de los empleados a comisión se obtiene multiplicando la cantidad de clientes captados por la comisión correspondiente a cada tipo de cliente. Si el monto por los clientes captados no llega a un monto mínimo (que es igual para todos los empleados a comisión y asciende a \$800.000), el empleado cobrará esta cantidad mínima.

- g.** Permite al usuario realizar las operaciones necesarias mediante un menú

2. Evaluar cuál será la salida por pantalla:

```
public abstract class A {
    public abstract void dinamica();
    public void estatica() {
        System.out.println( "Método estático de la clase A");
    }
}

public class B extends A {
    public void dinamica() {
        System.out.println( "Método dinámico de clase B");
    }
}

public class C extends A {
    public void dinamica() {
        System.out.println( "Método dinámico de clase C");
    }
}

public class Ligadura {
    static public void main(String [] ar){
        A a;
        B b = new B();
        C c = new C();
        System.out.print( "Métodos llamados con objeto b desde");
        System.out.println("referencia de la clase A");
        a = b;
        a.dinamica();
        a.estatica();
        System.out.print( "Métodos llamados con objeto c desde");
        System.out.println(" referencia de la clase A"); a = c;
        a.dinamica();
        a.estatica();
    }
}
```

- ¿Cual es la salida por pantalla?

3. **TestFlujo**: En la siguiente jerarquía de herencia existen llamadas a métodos en los que se debe tener claro el concepto de polimorfismo. Escribe cuál sería el resultado de la ejecución del siguiente código:

```
abstract class Flujo {  
  
    abstract public void escribe(char c);  
  
    public void escribe(String s) {  
        for (int i = 0; i < s.length(); i++) {  
            System.out.println("Escribe de Flujo ....");  
            escribe(s.charAt(i));  
        }  
    }  
  
    public void escribe(int i) {  
        escribe("" + i);  
    }  
}  
  
class Disco extends Flujo {  
  
    public void escribe(char c) {  
        System.out.println("Escribe de disco " + c);  
    }  
}  
  
class DiscoFlexible extends Disco {  
    public void escribe(String s) {  
        System.out.println("Escribe de Disco Flexible...");  
        super.escribe(s);  
    }  
}  
  
public class TestFlujo {  
  
    public static void main(String a[]) {  
        DiscoFlexible dc = new DiscoFlexible();  
        Flujo f = dc;  
        f.escribe("ab");  
    }  
}
```

```
}  
}
```

4. Detecta los dos errores que presenta este código y explica por qué no es correcto:

```
abstract class A {
```

```
    int i;
```

```
    A(int i) {
```

```
        this.i = i;
```

```
    }
```

```
}
```

```
class B extends A {
```

```
    void metodo() {
```

```
        System.out.println(i);
```

```
    }
```

```
}
```

```
class Cuestion {
```

```
    public static void main(String[] args) {
```

```
        A[] v = new A[10];
```

```
        for (int i = 0; i < 10; i++) {
```

```
            v[i] = new B();
```

```
            v[i].metodo();
```

```
        }
```